LLM API의 구조 (실습)

2-1. messages 배열과 role (실습)
2-2. 토큰과 컨텍스트 윈도우
2-3. 주요 파라미터 (실습)
2-4. LLM의 한계

2-1. messages 배열과 role (실습)

> system, user, assistant 메시지를 직접 바꿔 호출하며 동작 차이를 확인해봅니다.

채팅형 LLM API의 입력은 단일 문자열이 아니라 역할(role)이 붙은 메시지의 배열입니다.

- `system` : AI의 페르소나와 지침을 설정합니다. 보통 배열의 맨 앞에 둡니다.
- `user` : 사용자의 입력입니다.
- `assistant` : 모델의 응답입니다. 과거 응답을 대화 기록으로 다시 보낼 때 이 역할로 넣습니다.

왜 배열일까요. **모델은 상태를 저장하지 않기(stateless) 때문입니다.**

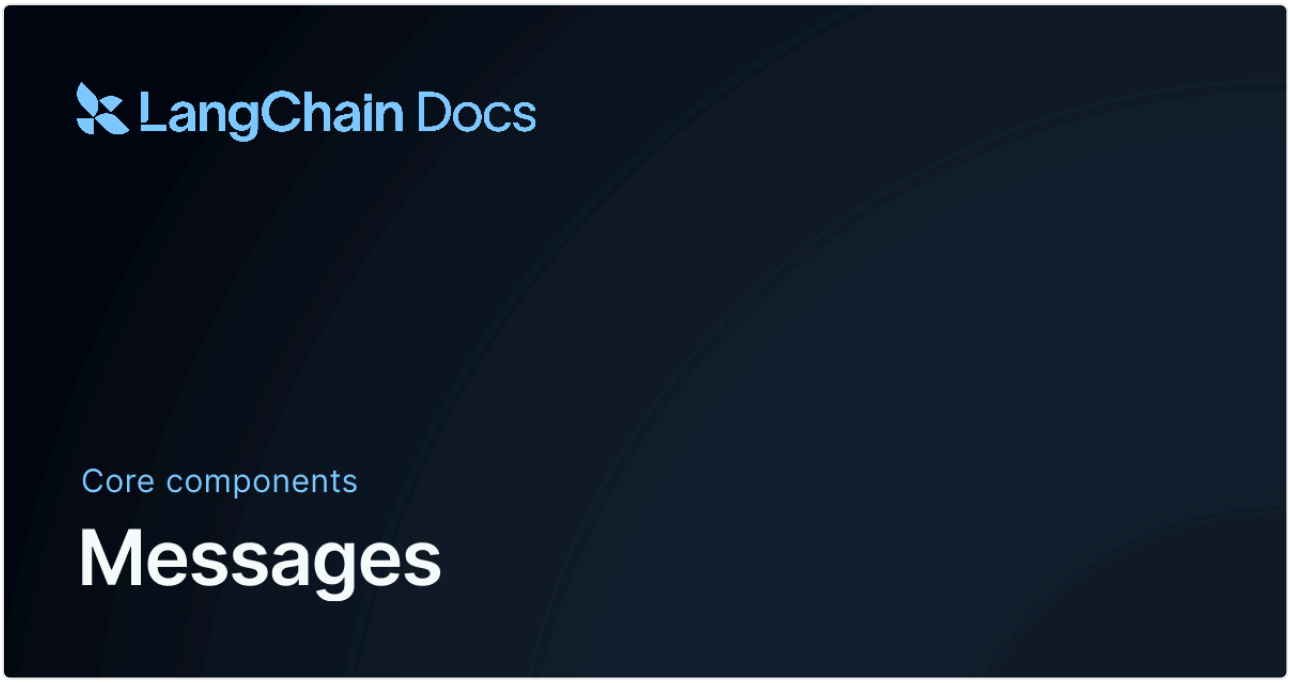
이전 대화 맥락을 알려면 과거 메시지를 매번 함께 보내야 하는데 그러려면 누가 한 말인지 구분되는 리스트 구조가 필요합니다.

LangChain에서는 이 역할들이 `SystemMessage` , `HumanMessage` , `AIMessage` 클래스로 대응됩니다.

Messages - Docs by LangChain

Use this file to discover all available pages before exploring further.

https://docs.langchain.com/oss/python/langchain/messages



따라하기

1. system 메시지가 있을 때와 없을 때 그리고 대화 기록이 있을 때의 차이를 직접 확인해봅니다.
   아래 코드를 `day1_practice/messages_role.py` 로 저장하고 실행합니다.

```
# messages_role.py - role별 동작 차이 확인
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

load_dotenv()
llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)

# 1) system 메시지 없이
r1 = llm.invoke([HumanMessage(content="사내 메신저가 접속이 안 됩니다.")])
print("[system 없음]", r1.content)

# 2) system 메시지로 페르소나 지정
r2 = llm.invoke([
    SystemMessage(content="너는 삼성SDS IT지원팀 상담원이야. 접수 확인과 예상 처리 절차를
정중한 존댓말 두 문장으로 안내해."),
    HumanMessage(content="사내 메신저가 접속이 안 됩니다."),
])
print("[system 있음]", r2.content)

# 3) 대화 기록(assistant 메시지 포함)으로 맥락 전달
r3 = llm.invoke([
    SystemMessage(content="너는 삼성SDS IT지원팀 상담원이야."),
    HumanMessage(content="사내 메신저가 접속이 안 됩니다."),
    AIMessage(content="접수되었습니다. 사용 중인 운영체제를 알려주시겠어요?"),
    HumanMessage(content="Windows 11이요."),  # 모델이 '메신저 장애' 맥락을 알고 답합
니다
])
print("[대화 기록]", r3.content)
```

   한 가지 미리 알고 가면 좋은 것이 있습니다. 보통은 `.content` 가 문자열로 깔끔하게 나오지만, `ChatBedrockConverse` 는 경우에 따라 `[{'type': 'text', 'text': '...'}]` 같은 **블록 리스트**로 돌려줍니다.
   하나의 응답을 여러 조각(텍스트, 도구 호출 요청 등)으로 나눠 보낼 수 있게 만든 구조여서, 도구 호출이 섞인 응답에서 주로 나타납니다. 리스트가 찍혀도 잘못한 것이 아니니 그대로 진행하면 되고, 2일차부터는 두 형태를 모두 문자열로 정리해 주는 `get_text` 헬퍼를 만들어 씁니다.

확인: 같은 질문인데 system 메시지 유무에 따라 답의 형식이 달라지고 대화 기록을 넣었을 때 이전 턴의 맥락을 이어받은 것을 확인하였으면 잘 진행한 것입니다!

2-2. 토큰과 컨텍스트 윈도우

> 사용량 응답 필드로 토큰 개념과 컨텍스트 한계를 이해해봅니다.

모델은 텍스트를 글자 단위가 아니라 토큰(token) 단위로 처리합니다.

토큰이 중요한 이유는 두 가지입니다.
하나는 과금 기준으로, API 요금은 입력 토큰 수 + 출력 토큰 수로 계산됩니다.
다른 하나는 컨텍스트 윈도우(context window)입니다. 모델이 한 번에 볼 수 있는 토큰 수의 상한으로 입력과 출력을 합쳐 이 한도를 넘길 수 없습니다.

멀티턴 대화에서 기록을 계속 쌓아 보내면 입력 토큰이 턴마다 누적됩니다. 이는 비용이 늘어나는 것과 제대로된 결론을 내지 못하고 결국 한도에 부딪히는 상황이 오게 됩니다.

따라하기

1. 아래 코드를 `day1_practice/tokens.py` 로 저장하고 실행해 응답의 토큰 사용량을 확인합니다.

```
# tokens.py - 응답의 토큰 사용량 확인
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse

load_dotenv()
llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)

result = llm.invoke("LangChain을 한 문장으로 설명해줘.")

print("응답:", result.content)

# 사용량은 호출 방식에 따라 안 실려 올 수 있으므로 먼저 있는지 확인합니다
if result.usage_metadata:
    usage = result.usage_metadata
    print(f"입력 토큰: {usage['input_tokens']}")
    print(f"출력 토큰: {usage['output_tokens']}")
    print(f"전체 토큰: {usage['total_tokens']}")
else:
    print("이번 응답에는 토큰 사용량이 실려 오지 않았습니다.")
```

2. 프롬프트를 길게 바꿔 입력 토큰이 어떻게 늘어나는지 비교해 봅니다.

막힐 때

- `usage_metadata` 가 `None` 입니다.
  호출 방식에 따라 사용량이 안 실려 오는 경우가 있습니다. `if result.usage_metadata:` 로 방어적으로 확인하는 습관을 들입니다.
- 한국어 프롬프트가 생각보다 토큰을 많이 쓰는건 정상입니다. 토크나이저가 영어 중심으로 학습되어 한국어는 글자당 토큰 수가 많습니다. 비용을 추산할 때 해당 내용을 염두에 두어 조금 더 버퍼를 두는 것이 좋습니다.

확인: `usage_metadata` 에서 입력, 출력, 전체 토큰 수를 읽을 수 있으면 성공입니다.

2-3. 주요 파라미터 (실습)

> temperature와 max_tokens를 바꿔가며 출력 차이를 비교해봅니다.

모델 호출 시 자주 조절하는 파라미터는 두 가지입니다.

- `temperature` : 다음 토큰을 뽑을 때의 무작위성입니다. 0에 가까우면 매번 거의 같은 답을 내고 1 근처면 다양한 답을 냅니다. 분류나 정보 추출처럼 일관성이 중요한 작업은 낮게, 아이디어와 같이 창의성이 높은 생성을 희망할 시 높게 설정합니다.
- `max_tokens` : 출력 토큰 수의 상한입니다. 비용과 응답 길이를 통제하며 너무 작게 잡으면 답이 중간에 잘립니다.

따라하기

1. 같은 프롬프트를 파라미터만 바꿔 반복 호출해 봅니다. 아래 코드를 `day1_practice/params.py` 로 저장하고 실행합니다.

```
# params.py - temperature와 max_tokens 비교
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse

load_dotenv()
MODEL = "us.anthropic.claude-sonnet-4-5-20250929-v1:0"

prompt = "삼성SDS 사내 카페 이름 아이디어를 하나만 제안해줘."

# temperature 0: 결정적인 출력. 반복 호출해도 거의 같은 답
llm_t0 = ChatBedrockConverse(model=MODEL, region_name="us-east-1", temperatu
re=0)
# temperature 1: 다양성 높은 출력. 호출마다 다른 답
llm_t1 = ChatBedrockConverse(model=MODEL, region_name="us-east-1", temperatu
re=1)

print("--- temperature=0 (3회 호출) ---")
for _ in range(3):
    print(llm_t0.invoke(prompt).content)

print("--- temperature=1 (3회 호출) ---")
for _ in range(3):
    print(llm_t1.invoke(prompt).content)

# max_tokens: 출력 길이 제한. 너무 작으면 답이 잘립니다
llm_short = ChatBedrockConverse(model=MODEL, region_name="us-east-1", max_to
kens=20)
result = llm_short.invoke("LangChain의 장점을 설명해줘.")
print("--- max_tokens=20 ---")
print(result.content)
print("종료 사유:", result.response_metadata.get("stopReason"))
```

종료 사유가 `end_turn` 이면 모델이 스스로 답을 끝낸 것이고 `max_tokens` 면 길이 한도에 걸려 잘린 것입니다. 답이 잘렸는지 판단할 때 이 필드를 봅니다.

막힐 때

- temperature 0인데도 답이 조금씩 다릅니다.
  LLM 서빙 특성상 temperature를 0으로 고정해도 출력이 완전히 똑같이 재현되지는 않습니다. 거의 같은 답이 나온다 정도로 이해하면 됩니다.

확인: temperature 0은 답이 거의 같고 1은 호출마다 달라지며 max_tokens=20에서 답이 잘리는 것을 보면 성공입니다.

> 같은 프롬프트라도 파라미터에 따라 출력이 달라진다는 걸 확인했습니다.

2-4. LLM의 한계

> 환각, 계산 취약, 상태 망각을 직접 재현해봅니다.

LLM 단독 호출로는 풀 수 없는 문제들이 있습니다.

1. 환각(hallucination): 모델이 틀린 내용을 자신 있게 답합니다. 학습 데이터에 없는 사내 정보를 물으면 그럴듯한 거짓을 지어냅니다. (요즘은 모델 학습개선 및 웹서치를 통해 많이 보완되긴 했습니다.)
2. 계산 취약: 정확한 집계 같은 산술에 약합니다. (소수점 5자리 이상 등)
3. 상태 망각(stateless): 호출이 끝나면 다 잊습니다.

사내 지식 부족과 환각은 RAG(2일차), 최신 정보와 계산은 Tool과 MCP(3~4일차), 상태 망각은 Message History(오늘 5-3)와 체크포인터(5일차)로 보완합니다.

따라하기

1. 아래 코드를 `day1_practice/limits.py`로 저장하고 실행해 세 가지 한계를 직접 재현해 봅니다.

```
# limits.py - LLM 한계 재현
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse

load_dotenv()
llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
    temperature=0,
)

# 1) 환각: 존재하지 않는 사내 규정을 물으면 지어낼 수 있습니다
print(llm.invoke("삼성SDS 연차 이월 규정 제12조 3항 내용을 알려줘.").content)

# 2) 계산 취약: 큰 수 곱셈
print(llm.invoke("1234567 곱하기 8901234는? 숫자만 답해.").content)
print("실제 답:", 1234567 * 8901234)

# 3) 상태 망각: 두 번의 독립 호출은 서로를 모릅니다
llm.invoke("내 이름은 김하늘이야.")
print(llm.invoke("내 이름이 뭐였지?").content)
```

```
회사의 내부 규정은 기밀 정보이거나 임직원에게만 공개되는 경우가 많아, 공개된 정보로는 확인이 어렵습니다.

도움이 더 필요하시면 말씀해 주세요!
10989611875778
실제 답: 10989169755678
죄송하지만, 저는 당신의 이름을 알지 못합니다. 우리가 이전에 대화를 나눈 적이 있다면, 저는 이전 대화 내용을 기억하지 못합니다. 각 대화는 새롭게 시작되기 때문입니다.

이름을 알려주시면 대화 중에 그렇게 불러드릴 수 있습니다. 어떻게 불러드리면 될까요?
```

막힐 때

- 모델이 모른다고 잘 답하긴 합니다. 최근 모델은 모른다고 잘 말하는 편이지만 100% 확실하게 모른다고 말하지는 않습니다. 구체적인 조항 번호를 지정하는 식으로 질문을 바꾸면 지어내는 경우를 재현해볼 수도 있습니다.
- 계산을 맞히는 경우도 있습니다. 자릿수를 더 키우거나 소수점 자릿수를 늘리면 실패율이 올라갑니다.

확인: 계산 결과가 실제 답과 다르고 두 번째 호출이 이름을 기억하지 못하는 것을 보면 성공입니다.